AI WEATHERWISE

Intelligent Weather Forecasting and AI-Powered Weather Insights

PHASE 6 – PROJECT TESTING

Project type: AI-powered weather forecasting and recommendation REST API / backend

Stack: Node.js · Express.js · MongoDB Atlas · Mongoose · JWT · bcrypt.js · Google Gemini AI · OpenWeatherMap · REST APIs · Thunder Client / Postman

Backend server: http://localhost:5001

BSc Computer Science – Final Year Project

Phase 6 Documentation: Test Plan, Test Cases and Test Evidence

1. Test Plan

1.1 Testing Objective

The purpose of testing is to verify that the AI WeatherWise REST API works as designed. Testing compares the expected behavior of each endpoint with the actual HTTP status code and JSON response returned by the running backend. The objective covers the following areas:

- **User registration and login:** users can create an account and authenticate securely.
- **JWT authentication and protected route security:** protected routes accept a valid token and reject missing, invalid or malformed tokens.
- **Weather data retrieval and Weather API integration:** the backend returns current weather for a requested city through the OpenWeatherMap integration, or through the fallback mode where applicable.
- **Favorite location management:** an authenticated user can create, view, update and delete their own favorite locations, and cannot access those of other users.
- **AI weather summary and recommendation generation:** valid weather input reaches the Google Gemini service and an AI response is returned in the expected JSON structure.
- **Input validation, error handling and consistent JSON responses:** invalid input is rejected and every failure is returned in a consistent JSON format.

The quality of Gemini's generated wording is **not** evaluated. AI testing checks only that the AI service receives valid input and that the API returns the expected response structure, since generated text can vary from request to request.

1.2 Testing Scope

In Scope

The following areas of the implemented AI WeatherWise API are tested. Only endpoints documented in the project source are named; where the source does not specify a path, the test is described generically.

- **Authentication APIs:** `POST /api/auth/register` and `POST /api/auth/login`, including JWT issuing and JWT-protected routes.
- **Weather API:** `GET /api/weather/:city` (public), including live retrieval and fallback (mock) behavior.
- **Favorite Location APIs:** create, read/list, update and delete operations on favorite locations (paths not specified in the source; specific-location retrieval only if implemented).
- **AI API:** `POST /api/ai/weather-recommendation`, and AI weather summary generation, which the source describes as part of the same AI Service integration.
- **Validation, authentication checks and error handling:** invalid requests, HTTP status codes and the JSON response format.

Out of Scope

- Front-end / user-interface testing, because AI WeatherWise is a REST API backend with no front end.
- Load and performance testing, because no such testing is part of the implemented project.
- Evaluating the subjective quality or accuracy of Gemini-generated text.

- Production infrastructure and deployment testing, because the project is tested in a local development environment.
- Testing the internal accuracy of OpenWeatherMap's data; only the backend's handling of the returned data is tested.

1.3 Testing Environment

Testing is performed against the backend running locally. Test accounts and test locations are used; no real sensitive personal information is entered.

Table 1: *AI WeatherWise testing environment*

Component	Details
Runtime	Local Node.js environment (v16 or later)
Backend framework	Express.js application started with the development script (nodemon)
Backend URL	http://localhost:5001 (PORT 5001)
Database	MongoDB Atlas accessed through Mongoose (User and Location collections)
Weather service	OpenWeatherMap API; fallback (mock) mode when the weather API key is not configured
AI service	Google Gemini AI through the @google/genai SDK
Configuration	.env file: PORT, MONGO_URI, JWT_SECRET, GEMINI_API_KEY, plus the OpenWeatherMap key when live weather is tested
Test client	Thunder Client (VS Code) or Postman
Test data	Dedicated test accounts (at least two, for access-control tests) and sample cities

Secrets such as the JWT secret, MongoDB connection string, Gemini API key and weather API key are supplied only through the .env file and are never written into this document.

1.4 Testing Tool

Testing is carried out with **Thunder Client / Postman**. Requests are saved in a collection (the project also contains a `postman_collection.json` file) organized into folders:

- **Authentication:** register and login requests
- **Weather:** current-weather requests for several cities
- **Favorite Locations:** create, list, update and delete requests
- **AI:** weather recommendation and weather summary requests
- **Errors / Validation:** invalid input, missing or invalid JWT and unknown route requests

JWT-protected requests are tested using the token returned by the login request. The token is stored in a client environment variable and sent as an Authorization header:

Table 2: *JWT testing flow*

Login request	→	**JWT token returned**	→	**Authorization: Bearer <token>**	→	**Protected API**

Actual JWT tokens and API keys are never shown in this report.

1.5 Testing Approach

AI WeatherWise is tested using **manual black-box testing**: the tester sends a request with known input, the API processes it, and the returned HTTP status code and JSON response are compared with the expected result. Each case is then recorded as Pass or Fail.

Table 3: *Black-box testing flow*

Send request	→	**Known input**	→	**API processes**	→	**Status + JSON**	→	**Compare expected vs actual**	→	**Pass / Fail**

The recommended execution order is:

1. Authentication (a valid user and JWT are needed for protected routes)
2. Weather API (public endpoint)
3. Favorite location CRUD and access control
4. AI weather summary and recommendation
5. Error and validation cases

Each testing category is described below in the context of AI WeatherWise.

- **1. Functional API testing:** checks that each endpoint performs its intended function with valid input.
- **2. Authentication testing:** checks registration, login, JWT issuing and the rejection of missing, invalid or malformed tokens.
- **3. Weather API testing:** checks weather retrieval for valid and invalid cities, the public access rule, the fallback (isMock) behavior and handling of external service failure.
- **4. Favorite location CRUD testing:** checks that locations can be created, read, updated and deleted, that duplicates are prevented, and that users can only access their own locations.
- **5. AI integration testing:** checks that valid weather input reaches Gemini AI and that missing input or AI failure is handled properly. Only the API behavior is checked, not the wording of the generated text.
- **6. Validation testing:** checks required fields, email format, password length, city/country values and location IDs.
- **7. Error handling testing:** checks that every error returns a suitable status code and a JSON error response.

1.6 Test Plan Summary

Table 4: *Test plan summary*

Test Area	Purpose	Expected Result
Authentication	Verify registration, login and JWT issuing	Valid users are created and can log in; invalid or duplicate data is rejected
Weather API	Verify current weather retrieval and OpenWeatherMap integration, including fallback mode	Valid city returns structured weather JSON; failures are handled safely
Favorite Locations	Verify create, read, update and delete of a user's locations	Authenticated user manages only their own locations; duplicates and invalid data rejected
AI Weather Summary	Verify AI summary generation through Gemini	Valid input returns a summary in the standard JSON format
AI Recommendation / Insights	Verify weather recommendation through `POST /api/ai/weather-recommendation`	Valid input returns a recommendation; missing input or AI failure is handled

Test Area	Purpose	Expected Result
Validation	Verify required fields, formats and IDs	Invalid input is rejected with a meaningful JSON message
Error Handling	Verify the centralized error handler	Correct status code and JSON error; no stack trace or secret exposed
JSON Response	Verify response format for success and failure	Every response is valid JSON in a consistent structure
Authorization	Verify JWT protection and per-user data access	Missing or invalid JWT is rejected; users cannot access other users' data

2. API Test Cases

This table gives endpoint-level coverage of the whole API. Detailed test tables for each module follow in Sections 3 to 7. Endpoint paths are taken from the AI WeatherWise source. Where the source does not specify a path (favorite locations and the AI weather summary), the path is described generically and not invented. Base URL: `http://localhost:5001`.

Table 5: *Endpoint-level API test cases*

Test ID	Module	Method	Endpoint	Test Scenario	Expected Result	Status
API-01	Authentication	POST	/api/auth/register	Register with valid name, email and password	201 Created; user object and JWT; no plain-text password	*Not Executed*
API-02	Authentication	POST	/api/auth/register	Register with an existing email	Rejected with JSON error	*Not Executed*
API-03	Authentication	POST	/api/auth/login	Login with valid credentials	200 OK; user object and JWT	*Not Executed*
API-04	Authentication	POST	/api/auth/login	Login with an incorrect password	Rejected; no JWT issued	*Not Executed*
API-05	Weather	GET	/api/weather/:city	Request weather for a valid city without a token	200 OK; city, temperature, humidity, windSpeed, condition	*Not Executed*
API-06	Weather	GET	/api/weather/:city	Request weather for an unknown city	Handled response (JSON error or documented fallback)	*Not Executed*
API-07	Locations	POST	Favorite location endpoint (path not specified in source)	Create a favorite location with a valid JWT	Location created and linked to the user	*Not Executed*
API-08	Locations	GET	Favorite location endpoint (path not specified in source)	List the user's favorite locations	Only the user's own locations returned	*Not Executed*
API-09	Locations	PUT	Favorite location endpoint (path not specified in source)	Update a favorite location	Record updated	*Not Executed*
API-10	Locations	DELETE	Favorite location endpoint (path not specified in source)	Delete a favorite location	Record removed	*Not Executed*
API-11	Locations	ANY	Favorite location endpoints	Call any operation without a JWT	Unauthorized JSON response	*Not Executed*
API-12	AI	POST	/api/ai/weather-recommendation	Request a recommendation with valid weather data	200 OK; AI recommendation in JSON	*Not Executed*
API-13	AI	POST	/api/ai/weather-recommendation	Request with missing input	Validation error	*Not Executed*
API-14	AI	POST	AI summary endpoint (not specified in source)	Request an AI weather summary	AI summary returned in standard JSON format	*Not Executed*
API-15	Errors	ANY	Protected route	Send an invalid JWT	Unauthorized JSON response	*Not Executed*
API-16	Errors	GET	Unknown route	Request a route that does not exist	JSON not-found style error	*Not Executed*

Status values: Pass, Fail or Not Executed. All cases are currently Not Executed.

3. Authentication Testing

Expected behavior: registration creates a user and never returns the plain-text password; duplicate emails and incomplete or invalid data are rejected; login returns a JWT only for correct credentials; protected routes respond only when a valid JWT is supplied. Registration and login are tested at `POST /api/auth/register` and `POST /api/auth/login`. The success codes confirmed in the source are 201 Created for registration and 200 OK for login; failure status codes are recorded during execution.

A. User Registration - POST /api/auth/register

Table 6: *User registration test cases*

Test ID	Scenario	Input	Expected Result	Status
AUTH-R01	Valid registration	name, valid email, password of 6+ characters	Registration succeeds (201 Created); user object and JWT returned; user document stored in MongoDB.	*Not Executed*
AUTH-R02	Duplicate email	Register again with an email that already exists	Request rejected with a JSON error message; no second user document is created.	*Not Executed*
AUTH-R03	Duplicate email in different letter case	Existing email re-submitted in upper case	Treated as a duplicate because emails are stored in lowercase and must be unique.	*Not Executed*
AUTH-R04	Missing name	email and password only	Rejected with a validation error; user not created.	*Not Executed*
AUTH-R05	Missing email	name and password only	Rejected with a validation error; user not created.	*Not Executed*
AUTH-R06	Missing password	name and email only	Rejected with a validation error; user not created.	*Not Executed*
AUTH-R07	Invalid email format	email without "@" or domain	Rejected by the email-format validation; user not created.	*Not Executed*
AUTH-R08	Password shorter than 6 characters	password of 5 characters	Rejected by the minimum-length validation; user not created.	*Not Executed*
AUTH-R09	Password handling	Inspect registration response and the users collection in MongoDB Atlas	Plain-text password never returned in the response; stored password differs from the submitted value (bcrypt hash).	*Not Executed*

B. User Login - POST /api/auth/login

Table 7: *User login test cases*

Test ID	Scenario	Input	Expected Result	Status
AUTH-L01	Valid credentials	Registered email and correct password	Login succeeds (200 OK); user object and JWT returned.	*Not Executed*
AUTH-L02	Incorrect password	Registered email, wrong password	Rejected with a JSON error message; no JWT issued.	*Not Executed*
AUTH-L03	Non-existing email	Email that is not registered	Rejected with a JSON error message; no JWT issued.	*Not Executed*
AUTH-L04	Missing email	password only	Rejected with a validation/error response; no JWT issued.	*Not Executed*
AUTH-L05	Missing password	email only	Rejected with a validation/error response; no JWT issued.	*Not Executed*

C. JWT Authorization

JWT tests use a protected favorite-location request and the public weather request as reference routes.

Table 8: *JWT authorization test cases*

Test ID	Scenario	Input	Expected Result	Status
AUTH-J01	Valid token on a protected route	Authorization: Bearer <valid token>	Protected request is processed normally.	*Not Executed*
AUTH-J02	Missing token	No Authorization header	Unauthorized JSON response; protected data not returned.	*Not Executed*
AUTH-J03	Invalid token	Bearer header with an altered or random token string	Unauthorized JSON response; protected data not returned.	*Not Executed*
AUTH-J04	Malformed Bearer header	Token sent without the "Bearer" prefix, or "Bearer" with no token	Unauthorized JSON response; request not processed.	*Not Executed*
AUTH-J05	Public route without token	GET weather request with no Authorization header	Request is processed (public route); no authentication error.	*Not Executed*

4. Weather API Testing

The AI WeatherWise backend receives a city name, communicates with the weather service (OpenWeatherMap) through the Weather Service layer, and returns the processed weather information as JSON. The endpoint is `GET /api/weather/:city` and, according to the source, it is **public**. When the external weather API key is not configured, the endpoint operates in a resilient fallback (mock) mode and flags its response with an isMock indicator. The response fields documented in the source are city, temperature, humidity, windSpeed and condition.

Expected behavior: a valid city returns weather JSON with status 200 OK without requiring a token; missing or invalid location input does not produce a server crash; external service failures are handled safely without exposing secrets.

Table 9: *Weather API test cases*

Test ID	Scenario	Input	Expected Result	Status
WTH-01	Valid weather request	GET /api/weather/:city with a valid city name (e.g. Chennai)	200 OK; JSON containing city, temperature, humidity, windSpeed and condition.	*Not Executed*
WTH-02	Second valid city	GET /api/weather/:city with a different valid city (e.g. London)	200 OK; JSON weather data for the requested city.	*Not Executed*
WTH-03	City name containing a space	City such as "New York" sent URL-encoded	Request handled correctly; weather JSON returned or a handled error if the city is not resolved.	*Not Executed*
WTH-04	Public access	Valid city, no Authorization header	200 OK; the public endpoint does not require a JWT.	*Not Executed*
WTH-05	Request with a JWT	Valid city, valid Bearer token	200 OK; same behavior as an unauthenticated request.	*Not Executed*
WTH-06	Missing city input	Weather URL sent without a city value	No weather data returned; JSON error (or not-found) response.	*Not Executed*
WTH-07	Invalid / unknown city	City name that does not exist (e.g. Xyzabc123)	Handled JSON response with a meaningful message; server does not crash. In fallback mode a structured mock response may be returned instead - record which mode was active.	*Not Executed*
WTH-08	Special characters in city	City value containing symbols or script-like text	Input handled safely; no server crash; JSON response returned.	*Not Executed*
WTH-09	JSON response format	Inspect a successful response	Response is valid JSON with Content-Type application/json and the fields city, temperature, humidity, windSpeed, condition.	*Not Executed*
WTH-10	Live weather mode	OpenWeatherMap key configured	Live data returned and not flagged as mock data.	*Not Executed*
WTH-11	Fallback (mock) mode	OpenWeatherMap key not configured	Structured weather response still returned (200 OK) with the isMock indicator.	*Not Executed*
WTH-12	External weather service failure	Invalid key or network unavailable for the weather service	Failure handled safely: fallback response or JSON error via the centralized handler; server keeps running.	*Not Executed*
WTH-13	Invalid weather API response	Unexpected or incomplete payload from the weather service (simulated where feasible)	Handled without crashing; no raw stack trace returned.	*Not Executed*
WTH-14	No secret exposure	Inspect response body and headers	Weather API key and other secrets are not present in the response.	*Not Executed*

5. Favorite Location Testing

The Location Controller manages a user's favorite locations. The source documents create, read, update and delete operations routed through `locationRoutes.js`, but does not give the endpoint paths, so paths are not stated in this document; the correct paths must be taken from the project routes when the requests are created in Thunder Client / Postman. All favorite-location operations require a valid JWT.

Expected behavior: an authenticated user can manage their own favorite locations; invalid input is rejected; unauthorized users cannot use location operations; invalid or non-existing records return appropriate JSON errors; and one user cannot access another user's locations.

Table 10: *Location model fields under test*

Field	Type	Description
`_id`	ObjectId	Unique identifier generated by MongoDB
`user`	ObjectId (ref: User)	Owner of the favorite location; set from the authenticated user
`city`	String (required)	Saved city name
`country`	String (required)	Country of the saved city
`createdAt`	Date	Creation timestamp (Mongoose timestamps)
`updatedAt`	Date	Update timestamp (Mongoose timestamps)

A compound unique index on user, city and country prevents the same user from saving the same city and country twice.

Table 11: *Favorite location test cases*

Test ID	Scenario	Input	Expected Result	Status
LOC-01	Add a favorite location	Valid JWT; city and country (e.g. Chennai, India)	Location created; response contains _id, user, city, country and createdAt; user equals the authenticated user.	*Not Executed*
LOC-02	Add a second favorite location	Valid JWT; different city and country	Second location created for the same user.	*Not Executed*
LOC-03	List favorite locations	Valid JWT	Only the authenticated user's locations are returned.	*Not Executed*
LOC-04	Get a specific location (if implemented)	Valid JWT; valid location ID	The matching location is returned; skip if the route is not implemented.	*Not Executed*
LOC-05	Update a location (if implemented)	Valid JWT; valid location ID; changed city and/or country	Record updated and the updated document returned.	*Not Executed*
LOC-06	Update with invalid ID	Valid JWT; malformed location ID	Rejected with a JSON error; no record changed.	*Not Executed*
LOC-07	Delete a favorite location	Valid JWT; ID of an owned location	Location removed; it no longer appears in the list.	*Not Executed*
LOC-08	Delete a non-existing location	Valid JWT; well-formed ID that is not in the database	Not-found style JSON error; no record affected.	*Not Executed*
LOC-09	Missing city	Valid JWT; country only	Rejected with a validation error; location not created.	*Not Executed*
LOC-10	Missing country	Valid JWT; city only	Rejected with a validation error; location not created.	*Not Executed*
LOC-11	Empty request body	Valid JWT; body {}	Rejected with a validation error; location not created.	*Not Executed*

Test ID	Scenario	Input	Expected Result	Status
LOC-12	Duplicate favorite by same user	Same city and country already saved by this user	Rejected (compound unique index on user, city, country); no duplicate document.	*Not Executed*
LOC-13	Same city saved by a different user	Second test account saves the same city and country	Allowed, because uniqueness is scoped to the user.	*Not Executed*
LOC-14	Invalid location ID format	Valid JWT; ID such as "abc123"	Invalid ID handled by the centralized error handler; JSON error returned.	*Not Executed*
LOC-15	No JWT	Create, list, update and delete each sent without a token	Unauthorized JSON response for every operation.	*Not Executed*
LOC-16	Invalid JWT	Create and list sent with an invalid token	Unauthorized JSON response; no data returned.	*Not Executed*
LOC-17	Access control - list	User B lists locations after User A has saved some	User B receives only User B's own locations (User A's are not visible).	*Not Executed*
LOC-18	Access control - update / delete	User B sends update and delete for a location owned by User A	Request rejected or not found; User A's record remains unchanged.	*Not Executed*
LOC-19	Ownership cannot be overridden	Body includes a user value belonging to another account	Stored user is the authenticated user, not the value supplied in the body.	*Not Executed*
LOC-20	Database verification	Inspect the locations collection in MongoDB Atlas after LOC-01 to LOC-07	Documents match the API responses; only the fields of the Location model are stored.	*Not Executed*

6. AI Testing

AI tests check the API behavior only. Gemini output varies from request to request, so the expected result is that a summary or recommendation is returned in the standard JSON format; the wording is never compared. Each test verifies that the request is accepted, the Gemini service is called, an API response is returned, the success or error structure is correct, and failures are handled properly. Include the JWT in the request if the AI routes are protected in the implementation; the source does not confirm this, so the authentication check is recorded during execution.

A. AI Weather Summary

The source describes AI weather summaries as part of the AI Service integration but does not list a separate endpoint for them. The summary tests are therefore defined against the AI summary functionality, and the endpoint used must be confirmed from the project routes. Input values: city, temperature, humidity, condition.

Table 12: *AI weather summary test cases*

Test ID	Scenario	Input	Expected Result	Status
AI-S01	Valid weather information	city, temperature, humidity, condition	200 OK; an AI weather summary is returned in the standard JSON format (wording not evaluated).	*Not Executed*
AI-S02	Missing required input	Empty request body	Validation error returned; the Gemini service is not called.	*Not Executed*
AI-S03	One required value missing	Any one of city, temperature, humidity or condition omitted	Validation error returned; no summary generated.	*Not Executed*
AI-S04	Invalid input values	Non-numeric temperature or humidity	Rejected with a JSON error where type validation is implemented; otherwise record the actual behavior.	*Not Executed*
AI-S05	Gemini API success	Valid input with a working GEMINI_API_KEY	Request reaches the AI Service, Gemini responds and the API returns the success structure.	*Not Executed*
AI-S06	Gemini API failure	Invalid key or Gemini unreachable (simulated in a test environment)	Error passed to the centralized error handler; JSON error returned; server keeps running.	*Not Executed*
AI-S07	Service failure / timeout	AI service call fails or times out	Handled safely with a JSON error response; no stack trace or key exposed.	*Not Executed*
AI-S08	Authentication check	Same request with and without a valid JWT	If the route is protected, no token returns Unauthorized and a valid token succeeds; if public, both succeed. Record the actual design.	*Not Executed*
AI-S09	Consistent structure	Send the identical request twice	Wording may differ, but both responses use the same JSON structure.	*Not Executed*

B. AI Weather Recommendation / Insight - POST /api/ai/weather-recommendation

Input fields: city, temperature, humidity, condition. A valid request asks Gemini AI for a personalized, condition-aware recommendation; missing input returns a validation error; and an AI or service failure is passed to the centralized error handler. The success status confirmed in the source is 200 OK.

Table 13: *AI weather recommendation test cases*

Test ID	Scenario	Input	Expected Result	Status
AI-R01	Valid weather data	POST /api/ai/weather-recommendation with city, temperature, humidity, condition	200 OK; an AI-generated recommendation is returned in the expected JSON structure.	*Not Executed*
AI-R02	Different weather conditions	Hot/humid, rainy and cold input values sent in turn	Each request returns 200 OK with a recommendation present; exact wording is not compared.	*Not Executed*
AI-R03	Missing all input	Empty request body	Validation error returned; Gemini not called.	*Not Executed*
AI-R04	Missing city	temperature, humidity, condition only	Validation error returned; no recommendation generated.	*Not Executed*
AI-R05	Missing condition or temperature	One of these values omitted	Validation error returned; no recommendation generated.	*Not Executed*
AI-R06	Invalid input types	Text where numbers are expected, or non-string city	Rejected with a JSON error where validation exists; otherwise record actual behavior.	*Not Executed*
AI-R07	Gemini API / service failure	Invalid key or unreachable service (simulated)	Failure handled by the centralized handler; JSON error; server stays available.	*Not Executed*
AI-R08	Authentication check	Request with and without a JWT	Behavior follows the route design (protected or public); record the actual result.	*Not Executed*
AI-R09	No secret exposure	Inspect response body and headers	GEMINI_API_KEY and other secrets are not present.	*Not Executed*
AI-R10	Repeated request	Same input sent twice	Structure identical; wording may vary and is not treated as a failure.	*Not Executed*
AI-R11	Chained flow	Use values from GET /api/weather/:city as the AI request input	200 OK; recommendation returned for the retrieved weather values.	*Not Executed*

7. Error and Validation Testing

7.1 Centralized Error Handling

Controllers and services do not build error responses individually. When something fails, the error is passed on and a single centralized error-handling mechanism converts it into a JSON response with an appropriate HTTP status code and a meaningful message. This gives every failure the same format. The errors covered by testing are listed below.

Table 14: *Errors handled by the centralized error handler*

Error	Typical Cause	Expected Handling
Validation error	Missing or invalid name, email, password, city, country or AI input	JSON error with a validation message
Duplicate email	Registering an email that already exists	JSON error; user not created
Missing JWT	Protected route called without a token	Unauthorized JSON error
Invalid JWT	Altered, malformed or wrong token	Unauthorized JSON error
Invalid location ID	Malformed or non-existing location ID	JSON error; no record changed
Database error	MongoDB unavailable or query failure	JSON error without connection details
Weather API failure	OpenWeatherMap error or network failure	Fallback or JSON error; server keeps running
Gemini AI failure	Invalid key, service error or timeout	JSON error; server keeps running
Unknown route	Request to a path that is not defined	JSON not-found error
Invalid request data	Malformed JSON or unexpected body	JSON error with a client-error status

Every error response must contain the appropriate HTTP status, a JSON body and a meaningful message. Error responses must **not** expose stack traces, database credentials, the JWT secret, the Gemini API key or the weather API key.

7.2 Error Test Cases

Exact failure status codes are not documented in the source, so the Expected Status column gives the class of status expected (for example 4xx) and the exact code must be recorded during execution. The only codes confirmed by the source are 201 Created (registration) and 200 OK (login, weather and AI success). The unknown-route 404 is the standard expectation for an undefined route.

Table 15: *Error test cases*

Test ID	Error Scenario	Input	Expected Status	Expected Response	Status
ERR-01	Missing required field	POST /api/auth/register without email	Client error (4xx)	JSON error message naming the validation problem	*Not Executed*
ERR-02	Duplicate email	POST /api/auth/register with an existing email	Client error (4xx)	JSON error message indicating the email already exists	*Not Executed*
ERR-03	Invalid login	POST /api/auth/login with a wrong password	Client error (4xx)	JSON error message; no token in the response	*Not Executed*
ERR-04	Missing JWT	Protected favorite-location request without Authorization header	Unauthorized (4xx)	JSON error message stating authentication is required	*Not Executed*
ERR-05	Invalid JWT	Protected request with an altered Bearer token	Unauthorized (4xx)	JSON error message stating the token is invalid	*Not Executed*
ERR-06	Invalid location ID	Location operation with a malformed ID such as "abc123"	Client error (4xx)	JSON error message; no stack trace	*Not Executed*

Test ID	Error Scenario	Input	Expected Status	Expected Response	Status
ERR-07	Invalid weather request	GET /api/weather/:city with a missing or unknown city	Client error (4xx) or handled response	JSON error message, or a documented fallback response	*Not Executed*
ERR-08	Weather service failure	Weather service key invalid or network blocked	Handled response or server error (5xx)	JSON response without secrets; server keeps running	*Not Executed*
ERR-09	Gemini service failure	POST /api/ai/weather-recommendation with the Gemini service unavailable	Server error (5xx)	JSON error message; no API key or stack trace	*Not Executed*
ERR-10	Unknown route	GET /api/unknown-route	Not found (404 expected)	JSON error response for the unknown route	*Not Executed*
ERR-11	Malformed JSON body	Register request with invalid JSON syntax	Client error (4xx)	JSON error message; server keeps running	*Not Executed*
ERR-12	Request sanitization	Login body with a database operator object in the email field	Client error (4xx) or sanitized input	No login succeeds; no operator is executed	*Not Executed*
ERR-13	Duplicate favorite location	Second create with the same city and country	Client error (4xx)	JSON error message indicating a duplicate	*Not Executed*
ERR-14	Database error	Database connection temporarily unavailable (simulated)	Server error (5xx)	JSON error message without connection string or credentials	*Not Executed*

8. Test Result Summary

The Total Test Cases column counts the planned cases in the detailed tables of Sections 3 to 7 (the endpoint overview in Section 2 refers to the same cases and is not counted again). No test has been executed yet, so Passed and Failed are 0 and every case is counted as Not Executed. The values must be **updated after execution** in Thunder Client / Postman: update each Status in the tables above, count them, and enter the final values below.

Table 16: *Test result summary (to be updated after execution)*

Test Category	Total Test Cases	Passed	Failed	Not Executed
Authentication	19	0	0	19
Weather API	14	0	0	14
Favorite Locations	20	0	0	20
AI Testing	20	0	0	20
Error and Validation	14	0	0	14
Total	87	0	0	87

9. Testing Flow Diagram

Figure 1 shows how each test request travels through the AI WeatherWise backend and how the result is judged. The tester prepares the request in Thunder Client / Postman; the Express server on PORT 5001 parses it, routes it, verifies the JWT on protected routes, and passes it through the controller and service layer to MongoDB Atlas, the weather service or Gemini AI. The response is compared with the expected result and recorded as Pass or Fail. Failures follow the error path through the centralized error handler and are re-tested after the issue is fixed.

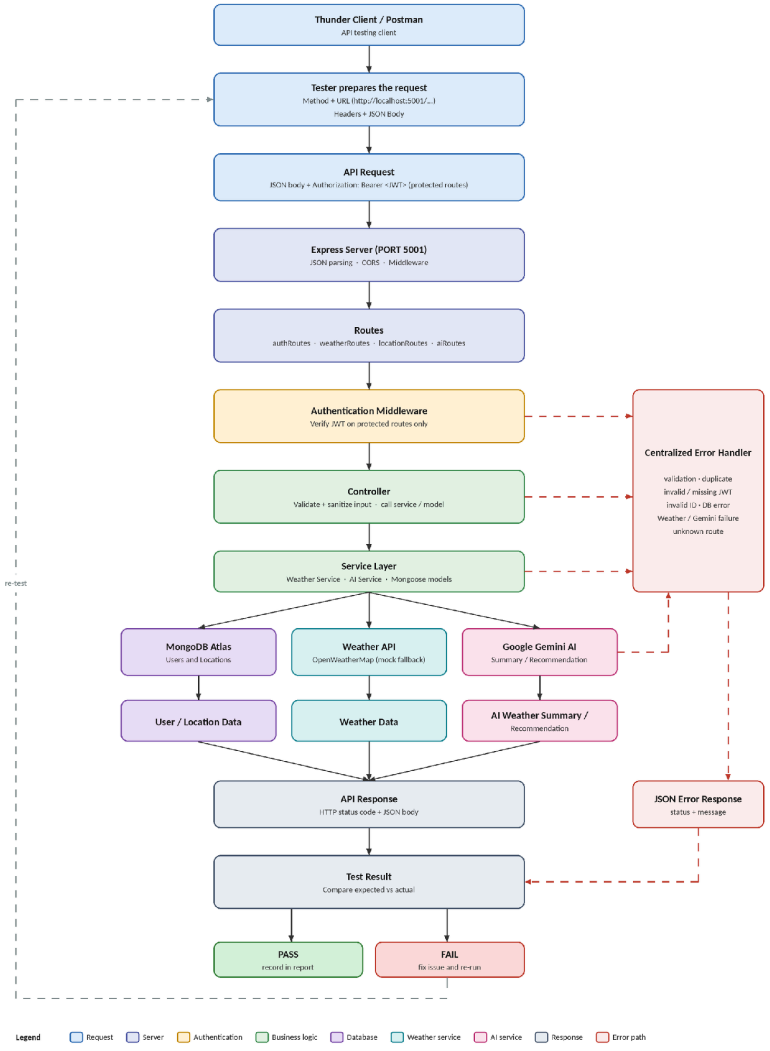


Figure 1: AI WeatherWise – API Testing Flow

(dashed red: error path; dashed grey: re-test after a failed result)

10. Phase 6 Conclusion

Testing is used to verify that AI WeatherWise works reliably and safely before it is presented. The test cases in this phase cover:

- **API functionality:** each endpoint behaves as designed
- **User registration and login:** account creation with hashed passwords and login with JWT issuing
- **JWT authentication:** protected route behavior and rejection of missing, invalid or malformed tokens
- **Weather data retrieval and Weather API integration:** current weather for a city, public access and fallback behavior
- **Favorite location operations:** creating, reading, updating and deleting locations, duplicate prevention and per-user access control
- **Gemini AI features:** weather summary and recommendation requests handled through the AI service
- **Input validation:** missing, invalid or unsupported data is rejected
- **Error handling:** centralized handling that returns suitable status codes without exposing secrets
- **JSON responses:** a consistent response format for both success and errors

Running these tests helps identify issues before the final demonstration and deployment, so that defects in authentication, weather retrieval, location management or AI integration can be corrected and re-tested. At the time of writing, the test cases are planned and have not been executed; the Status columns, the Test Result Summary and the screenshot evidence are to be completed from the actual Thunder Client / Postman runs against the backend on PORT 5001. Conclusions about the pass rate can be drawn only after those results are recorded.